

# L4-P01 练习手册

大模型开发基础

--环境搭建与推理实践

---

## 目 录

|                             |    |
|-----------------------------|----|
| 第一部分：如何使用 MoPaaS 平台.....    | 3  |
| 1. 登录平台.....                | 3  |
| 2. 创建密钥对（首次使用必做） .....      | 3  |
| 3. 准备文件存储 .....             | 3  |
| 4. 创建开发环境 .....             | 4  |
| 5. 使用开发环境 .....             | 4  |
| 6. 后续操作.....                | 5  |
| 7. 添加开放端口 .....             | 5  |
| 访问网络服务.....                 | 5  |
| 第二部分：大模型加载和使用 .....         | 6  |
| 实验 0：模型下载.....              | 6  |
| 实验 1：单轮对话（命令行） .....        | 6  |
| 使用框架.....                   | 6  |
| 安装依赖.....                   | 6  |
| 程序流程.....                   | 7  |
| 实验 2：多轮对话（命令行） .....        | 7  |
| 使用框架.....                   | 7  |
| 安装依赖.....                   | 7  |
| 程序流程.....                   | 8  |
| 实验 3：多轮对话（Web 界面） .....     | 8  |
| 使用框架.....                   | 8  |
| 安装依赖.....                   | 8  |
| 运行步骤.....                   | 8  |
| 程序流程.....                   | 9  |
| 访问地址.....                   | 9  |
| 实验 4：使用 vLLM 部署 API 服务..... | 10 |
| 安装依赖.....                   | 10 |
| 启动服务 .....                  | 10 |
| 调用示例（completions 接口） .....  | 10 |

## 第一部分：如何使用 MoPaaS 平台

1. 登录 → 2. 创建密钥 → 3. 文件存储 → 4. 创建环境 → 5. 使用环境 → 6. 后续运维

### 1. 登录平台

1. 打开浏览器，输入老师提供的 MoPaaS 平台地址。
2. 输入账号和密码，点击【登录】。
3. 登录后，点击右上角进入控制台。

### 2. 创建密钥对（首次使用必做）

1. 在左侧菜单找到 安全管理 → 密钥管理。
2. 点击【新建密钥对】，填写以下信息：
  - **名称**：必须用英文（如 student01）
  - **密码**：字母 + 数字组合（如 abc123）
3. 下载自动生成的密钥文件，妥善保存。

👉 以后创建环境都可以复用这个密钥，无需重复创建。

### 3. 准备文件存储

1. 在左侧菜单找到 我的资产 → 文件存储。
2. 点击【新建存储】，填写以下信息：
  - **名称**：使用英文（如 storage01）

- **容量**: 按需填写 (如 10 GB)
- **密钥对**: 选择刚才创建的密钥对

3. 点击【确定】，完成存储创建。

👉 上传的文件都会保存在此存储空间，建议提前规划好容量。

## 4. 创建开发环境

1. 在左侧菜单找到 模型开发 → 开发环境。
2. 点击【新建环境】，填写以下信息：
  - **环境名称**: 英文命名 (如 env01)
  - **密钥对**: 选择之前创建的密钥对
  - **平台镜像**: 选择 pytorch2.4 + GPU
  - **资源规格**: 按需选择 CPU / GPU / 内存
  - **存储**: 选择之前的存储，挂载目录须为英文路径
3. 点击【创建】，等待几分钟，直到环境状态变为【运行中】。

## 5. 使用开发环境

1. 点击环境右侧的【Mlab】，进入网页版 IDE。

- **Editor**: 编辑代码文件
- **Terminal**: 运行命令, 如 `pip install ...`、`python xxx.py`

2. 在 IDE 中进行实验: 运行代码、上传文件、调试模型等。

## 6. 后续操作

- **续约**：时间不够时，点击 **更多** → **续约**。
- **制作镜像（可选）**：安装完依赖后，可将环境保存为镜像，下次直接加载，避免重复安装。

## 7. 添加开放端口

路径：开发环境 → 详情 → 开放端口 → 添加端口

输入内部端口号后，系统会分配一个外部端口号（如 xxxxxx）。

## 访问网络服务

`http://223.113.240.48:xxxxxx`

💡 对外部用户开放 Web 应用时，请将 xxxxx 替换为系统分配的实际端口号。

## 第二部分：大模型加载和使用

本部分包含四个递进实验，涵盖从命令行推理到 Web 服务部署的完整流程：

| 实验编号 | 实验名称      | 核心技术                      | 输出方式     |
|------|-----------|---------------------------|----------|
| 实验 0 | 模型下载      | modelscope / pip          | 本地文件     |
| 实验 1 | 单轮对话（命令行） | Transformers / ModelScope | 终端输出     |
| 实验 2 | 多轮对话（命令行） | Transformers + colorama   | 彩色终端     |
| 实验 3 | 多轮对话（Web） | Gradio                    | 浏览器界面    |
| 实验 4 | API 服务部署  | vLLM                      | HTTP API |

### 实验 0：模型下载

本练习使用 Qwen1.5-1.8B-Chat 模型，通过以下命令下载：

```
pip install modelscope
modelscope download --model qwen/Qwen1.5-1.8B-Chat
```

📁 建议将模型放入 models 目录，例如：../models/Qwen1.5-1.8B-Chat

### 实验 1：单轮对话（命令行）

#### 使用框架

Transformers / ModelScope

#### 安装依赖

```
pip install torch transformers accelerate modelscope
```

📄 代码文件: `ex1_infer.py`

## 程序流程

1. 导入库：加载 modelscope（模型与分词器）和 torch（推理计算）。
2. 指定模型路径：设置模型名称或本地路径（如 ../models/Qwen1.5-1.8B-Chat）。
3. 加载分词器：使用 AutoTokenizer 将文本转换为 token。
4. 加载模型：使用 AutoModelForCausalLM 加载模型，并自动分配设备（GPU / CPU）。
5. 准备输入：例如："你好，请简单介绍一下你自己。"
6. 构造聊天模板：调用 tokenizer.apply\_chat\_template() 构造 Qwen 格式输入。
7. 转换为张量：生成 input\_ids 并移动到模型设备。
8. 生成回答：调用 model.generate()，生成约 100 个新 token。
9. 解码输出：将 token 转为文本，并去除特殊符号。
10. 输出结果：示例输出为 "助手：你好！我是 Qwen 智能助手，很高兴认识你。"

## 实验 2：多轮对话（命令行）

### 使用框架

Transformers / ModelScope

### 安装依赖

```
pip install torch transformers accelerate modelscope
```

📄 代码文件: `ex2_infer_chat.py`

## 程序流程

1. 设置编码：将标准输入输出设置为 UTF-8，避免中文乱码。
2. 导入与初始化：加载 modelscope、torch、colorama，支持彩色输出。
3. 加载模型与分词器：指定路径并自动检测设备（GPU / CPU）。
4. 初始化对话上下文：创建 messages 列表，设置 system 角色。
5. 定义对话函数 chat():
  - 接收用户输入，更新 messages
  - 构造模型输入，调用 model.generate()
  - 解码输出并追加到 messages
6. 主循环运行：
  - 显示欢迎信息，循环读取输入（exit / quit 退出）
  - 调用 chat() 输出结果，捕获异常（如显存不足）

## 实验 3：多轮对话（Web 界面）

### 使用框架

Gradio

### 安装依赖

```
pip install transformers accelerate gradio torch
```

📄 代码文件: `ex3_gradio_chat.py`

### 运行步骤



### 1. 启动程序：

```
python ex3_gradio_chat.py
```

### 2. 在开发环境中开放端口（如 7860）。

### 3. 浏览器访问：

```
http://223.113.240.48:xxxxxx
```

## 程序流程

### 1. 导入依赖：gradio（Web 界面）、modelscope（模型加载）、torch（推理计算）。

### 2. 加载模型：指定路径，加载 tokenizer 和模型。

### 3. 初始化对话历史：创建 messages，持续追加用户与模型的对话。

### 4. 定义核心函数 chat()：

- 输入：user\_input + history
- 构造模型输入 → 调用生成 → 更新对话历史 → 返回结果

### 5. 构建界面：包含标题（Markdown）、Chatbot、输入框、清除按钮。

- `msg.submit()` → 触发推理
- `clear.click()` → 清空对话

### 6. 启动服务：

```
demo.launch()
```

## 访问地址

- 本地： `http://127.0.0.1:7860`
- 外部： `http://223.113.240.48:xxxxxx`

## 实验 4：使用 vLLM 部署 API 服务

### 安装依赖

```
pip install vllm
```

### 启动服务

```
vllm serve ../models/Qwen1.5-1.8B-Chat \
  --served-model-name Qwen1.5-1.8B-Chat \
  --port 8000
```

### 调用示例 (completions 接口)

📄 代码文件: `ex4_vllm_request_completion.py`

```
import requests

url = "http://localhost:8000/v1/completions"
prompt = "写一首关于春天的现代诗"

response = requests.post(
    url,
    json={
        "model": "Qwen1.5-1.8B-Chat",
        "prompt": prompt,
        "max_tokens": 200,
        "temperature": 0.7
    }
)

print(response.json())
```

🚀 vLLM 提供 OpenAI 兼容的 API 接口，可直接使用 OpenAI SDK 调用，也支持批量推理，适合生产环境部署。

⚠️ 确保端口 8000 已在开发环境中开放，否则外部请求将无法访问。